



econ-radar

뉴스레터를 매일 도는 팀으로 만들었다

경제 뉴스를 매일 **3렌즈**로 정리해 **자동 발행**하고, 그 기록을 **리포트**, **블로그**, **책**으로 키우는 개인 지식 자산 시스템

다비코단 발표 · 2026-06-27 (v4) · 블로그, 텔레그램, 이메일 3채널

한 장 요약

- **문제** : 매일 같은 품질로 경제 뉴스를 정리하기. 사람이 직접 하면 지친다.
- **해법** : 일을 **에이전트 팀과 작업 매뉴얼(스킬)** 로 쪼개 매일 자동으로 돌린다.
- **차별점** : 한 번 쓰고 버리지 않는 **누적**. vault(옵시디언)에 쌓아 동향, 블로그, 책으로 키운다.
- **운영** : 매일 18:30 KST에 만들어 블로그, 텔레그램, 이메일로 완전 자동 발행. 승인 대신 **사실 검증(fact-check) 게이트**.

프롬프트 한 번이 아니라, 역할과 매뉴얼과 검수와 누적을 갖춘 **작은 조직을 설계했다**.

오늘 이야기할 네 가지

- **Part 1 무엇을, 왜** : 한 줄 정의, 실제 발행본, 3층 구조, 3렌즈
- **Part 2 어떻게 만들었나** : 8명의 에이전트 팀, 설계 4원칙, 파일로 둔 기준선, 자동화 3채널
- **Part 3 진화** : 17일간 운영과 배포가 설계를 고친 기록, 그리고 프로덕션이 가르친 것
- **Part 4 원칙, 교훈, 토의** : 안전선, 배운 점, 로드맵, 데모

슬라이드는 순서대로 넘기고, Q&A 때 "Part N"으로 돌아옵니다. 각 Part 앞에 구분 슬라이드가 있습니다.

Part 1

무엇을, 왜 만들었나

실제 발행본, 매일 저녁에 나간다



3층 구조, 오늘이 쌓여 책이 된다

층	무엇	산출물	자동화
1층 (매일)	수집→분석→편집→검수→검증→렌더	일일 뉴스레터, HTML, 푸시	매일 18:30 자동
2층 (자산)	옵시디언 vault에 누적	태그, <code>[[링크]]</code> , 주제 MOC	큐레이션 자동
3층 (저술)	쌓인 기록에서 패턴 추출	동향 리포트, 블로그, 책	주간 리포트 자동

- 1층은 매일 돌고, 2층은 연결하고, 3층은 매주 일요일에 끌어올린다.
- 위층으로 갈수록 기계가 만드는 건 초안까지다. 최종 판단과 발행은 사람이 한다.

3렌즈, 무엇을 보는가

뉴스를 늘 같은 세 각도로 뜯어본다.

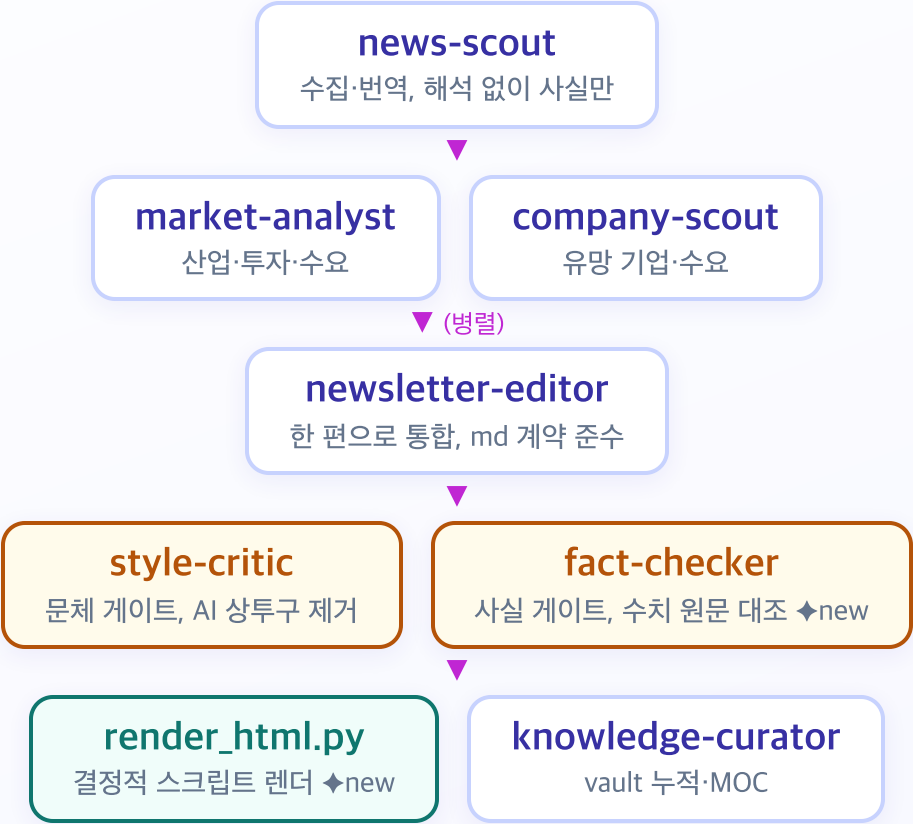
1. **산업구조 (Industry)** : 누가 움직였나, 밸류체인 어디가 바뀌나
2. **투자 (Investment)** : 테마, 종목, 리스크, 기회 (코스피와 해외)
3. **시장 (Market)** : 유망 기업(파는 쪽)과 **수요/Demand**(사는 쪽)

전 분야를 다루되 **제약·바이오·AI**에 무게를 둔다. 제약의 수요는 환자, 처방의, 지불자 셋으로 쪼갬다. 매수·매도 권유가 아니라 정보와 시나리오, 리스크로 다룬다.

Part 2

어떻게 만들었나: 하네스 구조

1층 파이프라인, 8명의 팀



설계의 핵심 4가지

- **역할 분리** : 수집가는 해석하지 않고, 분석가는 사실만 받고, 비평가는 표현만 고친다. 책임이 겹치지 않는다.
- **매뉴얼화(skill)** : 잘하는 법을 사람 머리가 아니라 `SKILL.md` 에 적어둔다. 그래서 매일 같은 품질이 나온다.
- **이중 게이트** : 발행 전에 `style-critic` (문체)과 `fact-checker` (사실)가 한 번씩 본다. 문체 게이트는 표현만, 사실 게이트는 수치와 출처만 본다.
- **판단만 LLM에** : HTML 렌더처럼 판단이 필요 없는 단계는 **스크립트로 고정한다**. 토큰을 아끼고 양식이 흐트러지지 않는다. 누적은 `knowledge-curator` 가 `[[링크]]` 와 MOC로 쌓는다.

품질을 어떻게 지키나: 기준선은 전부 파일

- **문체 기준선** : `korean-style-samples.md` (실제 기사 표본)를 꼭 참조한다. "A가 아니라 B", "~인 셈" 같은 AI 상투구는 금지.
- **사실 기준선** **NEW** : `fact-checker` 가 핵심 수치 최대 10개를 원문과 대조한다. 출처 없는 수치는 막고, 단일 소스는 표기하고, 1차 출처(IR·규제기관)는 끌어올린다.
- **깊이 기준선** : `benchmarks.md` 에 Why it matters 한 줄, 1차 자료 직접 링크, 양면 시각.
- **수요 렌즈** : `demand-lens.md` . 투자자는 공급과 자본, Demand는 수요로 본다.
- **양식 기준선** **NEW** : `NEWSLETTER-FORMAT.md` 계약과 템플릿 렌더로 양식이 흐트러지지 않는다.

기준선을 파일에 둔 게 핵심이다. 에이전트가 매번 같은 잣대를 본다.

자동화: 클라우드 루틴과 3채널 발행

루틴	언제	하는 일
데일리 완전 자동	매일 18:30 KST	수집→분석→편집→검수→검증→렌더→큐레이션→ 블로그·텔레그램·이메일
주간 동향	일요일 21:00 KST	한 주치에서 패턴 추출, <code>reports/</code> 동향 리포트(3층)

- **발행 채널 셋** : 블로그(아카이브), 텔레그램(@econradar, 저녁 즉시), **이메일** NEW (Buttondown). 트리거는 모두 git push 감지(GitHub Actions).
- **만든 시각과 배달 시각은 다르다** : 이메일은 저녁에 만들고 **다음날 아침 7시에 예약 발송한다**. 장 열기 전에 받아보게. 텔레그램은 저녁에 바로 나간다.
- 처음엔 발행 전에 사람이 승인했지만 **이틀 만에 접었다**. "확인 전까지 안 나간다"에서 "*"나가되 이상하면 내린다**로. 이메일만은 먼저 **초안(draft)**을 만든 뒤 확인한다.
- vault 전체를 git으로 추적한다(private). 클라우드가 과거를 읽고 여러 주에 걸친 흐름을 잇는다.

OKF: 지식을 파일로 쌓는 표준

2층 vault를 옵시디언이자 OKF(Open Knowledge Format)로 함께 굴린다.

- 파일 경로가 곧 개념 ID다. `topics/AI.md` 는 개념 'AI'. 폴더만 열면 그래프와 백링크가 산다.
- 모든 노트에 `type` 한 줄(daily, moc, analysis)과 `publish` 게이트. 공개는 명시한 노트만 나간다.
- 링크는 절대경로 마크다운 `[AI](/topics/AI.md)` 로 통일한다. 위키링크와 달리 어디서 열어도 안 깨진다.
- 디렉터리마다 `index.md` 가 진입점, 변경 이력은 `log.md` .

SDK도 DB도 없이 마크다운과 YAML만 쓴다. 사람도 에이전트도 `git clone` 하나로 읽는다.

Part 3

진화: 운영과 배포가 설계를 고민다

1주차: 5일간의 진화, 운영이 설계를 고친다

- **6/09** 하네스 구축, 첫 발행. 깊이가 부족하다는 피드백을 받고 항목 수와 Why it matters를 규칙으로 박았다
- **6/10** "문체가 AI 같다." 실제 기사 표본을 들고 style-critic 게이트를 달았다. 렌즈도 교체(커리어→유망기업)
- **6/11** "투자자 관점에 치우쳤다"는 스터디 피드백에 **수요(Demand) 렌즈**를 더했다. 클라우드 루틴 가동
- **6/12** 승인 게이트를 접고 **완전 자동 발행**으로. 텔레그램 채널 추가
- **6/13** 구조를 다시 보고 **4건을 한꺼번에 개선**: 사실 게이트, vault 전체 추적, 렌더 스크립트화, 주간 3층 루틴

패턴은 하나다. 어색하다, 치우쳤다 같은 막연한 불만을 파일로 된 규칙과 게이트로 바꾼다.

6/13 구조 검토, 비판이 곧 백로그

발행 4호 시점에 하네스를 스스로 비판해 보고, 우선순위 4건을 바로 적용했다.

발견한 문제	처방
게이트가 문체뿐, 잘못된 수치가 그대로 나간다	<code>fact-checker</code> 신설 (T4.7)
3층(저술)이 비어 있다, forcing function 부재	주간 동향 루틴 (일 21:00)
클라우드가 과거를 못 본다 (vault 로컬 전용)	레포 private, vault 전체 추적
렌더가 제일 비싼데(93K 토큰) 판단은 불필요	<code>render_html.py</code> 스크립트화

남은 건(MOC 요약 구조, 백로그 트리야지) TODO로 미뤘다. 검토 전문은 `vault/_meta/2026-06-13-harness-review.md`

2주차~3주차: 채널 확장과 프로덕션 (6/14 → 6/25)

- **6/14** 미뤄둔 백로그 처리. MOC 갱신 규칙, 렌즈 범위 분리, 승격 기준을 문서로 정리
- **6/15** 텔레그램 직접 발송이 네트워크에 막혀 **GitHub Actions(push 감지)로 발송을 옮겼다**
- **6/16-18** Actions 다중커밋 감지 버그를 고치고 **뉴스 신선도와 중복 방지** 추가. 같은 약과 기업이 며칠씩 반복되던 "에이전트 관성"을 걷어냈다
- **6/24** 오피디언에서 **OKF 포맷으로 마이그레이션**. 위키링크 1,228개 변환, frontmatter, index/log 정리
- **6/25 이메일 채널** 추가(Buttondown, 다음날 아침 예약). 발음, 버튼, 섹션 목차 등 프로덕션 다듬기

1주차 패턴이 그대로 이어진다. 운영과 배포가 새 실패 모드를 드러내면, 그게 규칙과 게이트, 채널로 바뀐다.

프로덕션이 가르친 것: 채널을 늘리며 부딪힌 벽

증상 (어디서)	진짜 원인	하네스 교훈
이메일 버튼·글씨가 깨짐	메일은 JS와 외부 CSS가 안 먹고, 클라이언트가 링크색을 덮어씀	로컬에서 된다고 받은편지함에서 되는 게 아니다. 인라인 CSS, table, <code>!important</code>
새 데일리가 옛 포맷으로 회귀	OKF 규약이 <code>~/.claude</code> 전역에만 있어 클라우드가 못 봄	규약과 기준선은 레포 안에 뒤야 유지된다
발음이 엉뚱한 단어	표제어 대신 괄호 속 영어 설명을 읽음	데이터에서 렌더로 가는 계약 을 명시(표제어 = 발음)
"고쳤는데 그대로"	배포 워크플로 지연에 브라우저·CDN 캐시까지	라이브는 curl로 검증하고 캐시를 의심한다

채널마다 런타임이 다르다. 같은 산출물도 **배포 환경에서 다시 검증해야 한다.**

운영 비용: 재 봐야 고친다

1회 발행에 약 43만 토큰, 벽시계로 25분쯤 든다 (`run-metrics.md` 에 단계별 누적)



가장 비싼 단계가 판단은 가장 필요 없는 단계였다. 스크립트로 갈아 발행당 약 22% 절감.

Part 4

원칙, 교훈, 토의

안전선: 하지 않는 것

- **자동 발행은 데일리에만** : 블로그 발행과 텔레그램 알림만 승인된 자동 동작이다. 메일과 메신저 대량 전송, 출판은 여전히 사람이 승인해야 나간다.
- **매수·매도 권유 금지** : 투자와 유망기업은 정보, 시나리오, 리스크로만 다룬다.
- **출처 없는 수치 생성 금지** : fact-checker가 출처 없는 수치를 발행 전에 막는다. 추정은 "추정"이라 밝힌다.

자동화의 신뢰는 무엇을 자동으로 하느냐보다 무엇을 자동으로 안 하느냐에서 온다.

하네스로 배운 점 (스터디 토의용)

- **일을 조직으로 본다** : 프롬프트 한 방이 아니라 역할과 매뉴얼, 검수, 누적을 갖춘 작은 팀이다.
- **지식은 파일에 둔다** : 기준선과 표본, 계약을 파일에 두면 품질이 사람에게 묶이지 않는다.
- **게이트는 실패 모드별로** : 문체 게이트는 어색함을, 사실 게이트는 오류를 막는다. 자동 발행일수록 사실 게이트가 먼저다.
- **판단과 기계 작업을 가른다** : 판단 없는 단계(렌더)는 스크립트로, 판단하는 단계만 LLM으로.
- **운영이 설계를 고친다** : 17일간 거의 매일 구조가 바뀌었다. 모두 운영하다 나온 불만에서 출발했다.
- **배달 환경마다 다시 검증한다** : 로컬 성공이 전부가 아니다. 이메일, 클라우드, 캐시는 저마다 다른 런타임이다.

→ 토의: 여러분의 반복 업무에서 "판단이 필요한 단계"와 "기계로 굳힐 단계"는 어디서 갈릴까요?

솔직한 미완: 다음 백로그

- **OKF 지속성** : 마이그레이션은 했지만 클라우드 에이전트가 아직 옛 포맷으로 만든다. 규약을 **레포 안으로** 옮겨야 자리잡는다(지금은 매일 클린업이 필요).
- **이메일 자동 발송 전환** : 지금은 초안 검토 단계. 안정되면 레포 변수 하나로 다음날 아침 자동 발송으로.
- **중복 규칙 검증** : "한 사건은 한 번만 서술" 규칙을 넣었으니, 실제 발행본에서 중복감이 줄었는지 며칠 더 본다.

하네스는 끝나는 게 아니라 **백로그가 도는 것이다**. 비판이 곧 다음 할 일이다.

직접 보기: 아카이브와 설계 노트

kakyungkim.github.io/econ-radar/



econ-radar

아카이브

제약·바이오·AI에 무게를 둔 경제 데일리 — 산업구조·투자·유망 기업 3렌즈
매일 신뢰 경제지를 스캔해 정리합니다. 아직 실험 단계예요.

발행본

2026-06-12

mRNA 암백신 상업화 문턱 · K-바이오 L/O 2.9조 · 코스피 8,000 돌파

2026-06-11

미 CPI·ECB 동시 분수령 · 오라클이 드러낸 AI 인프라 청구서

.../econ-radar-agent-harness/ (설계 노트 블로그)

Ka-Kyung Kim

Bioinformatics · Data Science · Life Science

Topics About 한국어 EN

Ka-Kyung Kim

Bioinformatics specialist — NGS pipelines, clinical data, genomics.

Seoul, Korea

Email

GitHub

LinkedIn

경제 뉴스 읽기를 포기하고 AI 에이전트 팀을 만들었다 — econ-radar 설계 노트

June 10, 2026 · 3 minute read

Read this post in English

안녕하세요. 임상 데이터, 유전체학, 머신러닝을 넘나들며 일하는 바이오인포매틱스 연구자 김가경입니다.

하지만 오래가지 못했습니다. 며칠은 열심히 읽다가도 어느 순간 흐름이 끊기곤 했습니다. 의지가 부족해서라기보다 정보량이 너무 많았습니다. 읽어야 할 기사가 너무 많고, 하나씩 정리하다 보면 시간이 훌쩍 지나곤 했습니다.

제가 포기한 건 경제 뉴스를 읽는 일이 아니라, 매일 직접 수집하고 정리하는 방식이었습니다. 그래서 문득 이런 생각이 들었습니다. “AI를 어떻게 쓸까”가 아니라 “이 반복 작업을 시스템으로 만들 수 있을까”로요. 뉴스를 모으고 정리하는 일은 역할을 나눈 에이전트 팀에 맡기고, 저는 판단과 검수에만 집중하기로 했습니다. 그렇게 해서 지금의 econ-radar가 만들어졌습니다.

econ-radar란?

econ-radar는 매일 주요 경제지를 스캔해 산업 구조, 투자 흐름, 시장이라는 세 가지 관점으로 정리하는 개인 경제 브리핑입니다.

On this page

econ-radar란?

하나의 AI가 아니라 적은 팀

만들면서 배운 점

1. “어색하다”는 피드백은 규칙이 되어야 한다

2. 사람이 할 일을 줄이는 게 목표다

3. 사실과 해석은 분리하는 편이 낫다

4. 읽고 끝나는 정보보다 쌓이는 정보

5. 누가 파는지는 봤지만, 누가 사는지는 보지 못했다

출발점

직접 보기

다음

▲ 클릭하면 각 페이지로 이동

2026-06-27 · 다비코단 발표

24

감사합니다

econ-radar : 매일 도는 3렌즈 뉴스 팀, 이중 게이트, 3채널 발행, 누적되는 지식 자산



QR은 아카이브로 · 발행 채널은 블로그, 텔레그램 @econradar, 이메일 셋 · 데모: 오늘자 HTML 리포트